

Mini-projets PowerShell complets, étape par étape, pour automatiser des tâches courantes d'administration système.

Mini-projet 1 : Surveillance du système avec alerte

Objectif : Récupérer périodiquement l'utilisation CPU, RAM et disque, écrire les valeurs dans un fichier CSV, et déclencher une alerte (fichier log + message console) si un seuil est dépassé.

Étape 1 : Créer le script SystemMonitor.ps1

Ouvrez VS Code ou PowerShell ISE, enregistrez le fichier.

Étape 2 : Récupérer les métriques

```
```powershell
Obtenir le pourcentage d'utilisation CPU

$cpu = (Get-Counter "\Processor(_Total)\% Processor
Time").CounterSamples.CookedValue

Obtenir la mémoire utilisée (en pourcentage)

$ram = (Get-Counter "\Memory\% Committed Bytes In
Use").CounterSamples.CookedValue

Obtenir l'espace disque libre sur C: (en pourcentage d'utilisation)

$disk = (Get-PSDrive C | Select-Object Used,
@{N="Percent";E=[math]::Round(($_.Used / ($_.Used + $_.Free)) * 100, 2)}).Percent
```
```

Étape 3 : Ajouter un horodatage et écrire dans un CSV

```
```powershell
$timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
$data = [PSCustomObject]@{
 Time = $timestamp
 CPU_Percent = [math]::Round($cpu, 2)
 RAM_Percent = [math]::Round($ram, 2)
 Disk_Percent = $disk
}

$csvPath = "C:\Logs\SystemMonitor.csv"
Créer le dossier si inexistant
$dir = Split-Path $csvPath -Parent
If (-not (Test-Path $dir)) { New-Item -ItemType Directory -Path $dir -Force }

Ajouter la ligne au CSV (créer l'en-tête si fichier nouveau)
If (-not (Test-Path $csvPath)) {
 $data | Export-Csv -Path $csvPath -NoTypeInfoInformation
} else {
 $data | Export-Csv -Path $csvPath -NoTypeInfoInformation -Append
}
```
```

Étape 4 : Vérifier les seuils et envoyer une alerte

```
```powershell
$seuilCPU = 80
$seuilRAM = 85
$seuilDisk = 90
```

```

If ($cpu -gt $seuilCPU -or $ram -gt $seuilRAM -or $disk -gt $seuilDisk) {
 $message = "ALERTE Système : "
 If ($cpu -gt $seuilCPU) { $message += "CPU=$cpu% > ${seuilCPU}%; " }
 If ($ram -gt $seuilRAM) { $message += "RAM=$ram% > ${seuilRAM}%; " }
 If ($disk -gt $seuilDisk) { $message += "DISK=$disk% > ${seuilDisk}%; " }
 Write-Host $message -ForegroundColor Red
 # Écrire dans un fichier d'alerte
 $alertLog = "C:\Logs\Alerts.log"
 "$timestamp - $message" | Out-File -FilePath $alertLog -Append
}
...

```

## Étape 5 : Assembler le script complet avec paramètres

```

```powershell
<#
.SYNOPSIS
    Surveille CPU, RAM, disque et alerte si seuils dépassés.
.DESCRIPTION
    À exécuter périodiquement via une tâche planifiée.
.PARAMETER CsvPath
    Chemin du fichier CSV de sortie (défaut: C:\Logs\SystemMonitor.csv)
.PARAMETER AlertLog
    Chemin du fichier d'alerte (défaut: C:\Logs\Alerts.log)
#>
Param(
    [string]$CsvPath = "C:\Logs\SystemMonitor.csv",
    [string]$AlertLog = "C:\Logs\Alerts.log"
)

```

```
# Seuils modifiables
```

```
$seuilCPU = 80
```

```
$seuilRAM = 85
```

```
$seuilDisk = 90
```

```
$timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
```

```
# Récupération métriques
```

```
Try {
```

```
    $cpu = (Get-Counter "\Processor(_Total)\% Processor Time" -ErrorAction  
Stop).CounterSamples.CookedValue
```

```
} catch { $cpu = $null }
```

```
Try {
```

```
    $ram = (Get-Counter "\Memory\% Committed Bytes In Use" -ErrorAction  
Stop).CounterSamples.CookedValue
```

```
} catch { $ram = $null }
```

```
Try {
```

```
    $disk = (Get-PSDrive C | Select-Object Used, Free)
```

```
    $diskUsedPercent = [math]::Round(($disk.Used / ($disk.Used + $disk.Free)) * 100, 2)
```

```
} catch { $diskUsedPercent = $null }
```

```
# Créer dossier parent si besoin
```

```
Foreach ($path in @($CsvPath, $AlertLog)) {
```

```
    $dir = Split-Path $path -Parent
```

```
    If ($dir -and -not (Test-Path $dir)) { New-Item -ItemType Directory -Path $dir -Force |  
Out-Null }
```

```
}
```

```
# Sauvegarde CSV
```

```
$data = [PSCustomObject]@{
```

```

Time = $timestamp

CPU_Percent = if ($cpu) { [math]::Round($cpu,2) } else { "N/A" }
RAM_Percent = if ($ram) { [math]::Round($ram,2) } else { "N/A" }
Disk_Percent = if ($diskUsedPercent) { $diskUsedPercent } else { "N/A" }
}

If (-not (Test-Path $CsvPath)) {
    $data | Export-Csv -Path $CsvPath -NoTypeInfoInformation
} else {
    $data | Export-Csv -Path $CsvPath -NoTypeInfoInformation -Append
}

# Alerte
$alertMsg = ""

If ($cpu -and $cpu -gt $seuilCPU) { $alertMsg += "CPU=$( [math]::Round($cpu,2))% >
${seuilCPU}%"; " }

If ($ram -and $ram -gt $seuilRAM) { $alertMsg += "RAM=$( [math]::Round($ram,2))% >
${seuilRAM}%"; " }

If ($diskUsedPercent -and $diskUsedPercent -gt $seuilDisk) { $alertMsg +=
"DISK=$diskUsedPercent% > ${seuilDisk}%"; " }

If ($alertMsg) {
    $fullAlert = "$timestamp – ALERTE : $alertMsg"
    Write-Host $fullAlert -ForegroundColor Red
    $fullAlert | Out-File -FilePath $AlertLog -Append
} else {
    Write-Host "$timestamp – Tout est normal." -ForegroundColor Green
}
...

```

Étape 6 : Planifier l'exécution automatique

Dans PowerShell administrateur :

```
```powershell
```

```
$action = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument "-File
C:\Chemin\Vers\SystemMonitor.ps1"
```

```
$trigger = New-ScheduledTaskTrigger -Daily -At "09:00"
```

```
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType
ServiceAccount
```

```
$settings = New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
DontStopIfGoingOnBatteries
```

```
Register-ScheduledTask -TaskName "SystemMonitor" -Action $action -Trigger $trigger -
Principal $principal -Settings $settings
```

```
```
```

Ou exécution toutes les 5 minutes :

```
```powershell
```

```
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-
TimeSpan -Minutes 5)
```

```
```
```

Mini-projet 2 : Gestionnaire de sauvegarde avec compression et rotation

Objectif : Sauvegarder récursivement un dossier source vers une destination avec horodatage, compression en ZIP, et suppression automatique des sauvegardes de plus de N jours.

Étape 1 : Définir les paramètres

```
```powershell
Param(
 [string]$Source = "C:\ImportantData",
 [string]$DestinationRoot = "D:\Backups",
 [int]$RetentionDays = 7,
 [switch]$Compress
)
```
```

Étape 2 : Vérifier l'existence du dossier source

```
```powershell
If (-not (Test-Path $Source)) {
 Write-Error "Le dossier source '$Source' n'existe pas."
 Exit 1
}
If (-not (Test-Path $DestinationRoot)) {
 New-Item -ItemType Directory -Path $DestinationRoot -Force | Out-Null
}
```
```

Étape 3 : Créer un dossier de sauvegarde horodaté

```
```powershell
$backupName = "Backup_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss')"
$backupPath = Join-Path $DestinationRoot $backupName
New-Item -ItemType Directory -Path $backupPath | Out-Null
Write-Host "Sauvegarde dans : $backupPath"
```
```

Étape 4 : Copier les fichiers (avec Robocopy pour la robustesse)

```
```powershell
$robocopyArgs = @($Source, $backupPath, "/MIR", "/R:3", "/W:10", "/NP", "/NDL")
$result = robocopy @robocopyArgs
If ($LASTEXITCODE -ge 8) {
 Write-Warning "La copie a rencontré des erreurs (code $LASTEXITCODE)"
} else {
 Write-Host "Copie terminée avec succès."
}
```
```

Étape 5 : Compresser en ZIP si demandé

```
```powershell
If ($Compress) {
 $zipFile = "$backupPath.zip"
 Compress-Archive -Path $backupPath -DestinationPath $zipFile -CompressionLevel
 Optimal
 Remove-Item -Path $backupPath -Recurse -Force
 Write-Host "Sauvegarde compressée : $zipFile"
}
```

```

 $finalPath = $zipFile
} else {
 $finalPath = $backupPath
}
...

```

## Étape 6 : Supprimer les anciennes sauvegardes (rotation)

```

```powershell
$limitDate = (Get-Date).AddDays(-$RetentionDays)
Get-ChildItem -Path $DestinationRoot | Where-Object {
    $_.Name -match "^Backup_" -and $_.CreationTime -lt $limitDate
} | ForEach-Object {
    If ($_.PSIsContainer) {
        Remove-Item -Path $_.FullName -Recurse -Force
        Write-Host "Supprimé ancien dossier : $($_.Name)"
    } else {
        Remove-Item -Path $_.FullName -Force
        Write-Host "Supprimé ancien ZIP : $($_.Name)"
    }
}
...

```

Étape 7 : Assembler le script complet BackupManager.ps1

```

```powershell
<#
.SYNOPSIS
 Sauvegarde un dossier avec horodatage, compression optionnelle, rotation.
.DESCRIPTION

```

Copie (mirror) le contenu de Source vers  
DestinationRoot\Backup\_AAAAMMJJ\_HHMMSS.

Si -Compress, archive en ZIP et supprime le dossier.

Supprime les sauvegardes (dossier ou ZIP) de plus de RetentionDays jours.

.PARAMETER Source

Dossier à sauvegarder (par défaut C:\ImportantData)

.PARAMETER DestinationRoot

Racine où stocker les sauvegardes (par défaut D:\Backups)

.PARAMETER RetentionDays

Nombre de jours de conservation (par défaut 7)

.PARAMETER Compress

Active la compression en ZIP

.EXAMPLE

.\BackupManager.ps1 -Source "C:\MesDocs" -DestinationRoot "E:\Archives" -  
RetentionDays 14 -Compress

#>

Param(

[string]\$Source = "C:\ImportantData",

[string]\$DestinationRoot = "D:\Backups",

[int]\$RetentionDays = 7,

[switch]\$Compress

)

# Vérifications

If (-not (Test-Path \$Source)) {

Write-Error "Le dossier source '\$Source' n'existe pas."; exit 1

}

If (-not (Test-Path \$DestinationRoot)) {

New-Item -ItemType Directory -Path \$DestinationRoot -Force | Out-Null

}

```
Création du dossier de sauvegarde
```

```
$backupName = "Backup_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss')"
```

```
$backupPath = Join-Path $DestinationRoot $backupName
```

```
New-Item -ItemType Directory -Path $backupPath | Out-Null
```

```
Write-Host "Sauvegarde vers : $backupPath"
```

```
Copie avec Robocopy (mirror)
```

```
$robocopyArgs = @($Source, $backupPath, "/MIR", "/R:3", "/W:10", "/NP", "/NDL")
```

```
$result = robocopy @robocopyArgs
```

```
If ($LASTEXITCODE -ge 8) {
```

```
 Write-Warning "Erreurs lors de la copie (code $LASTEXITCODE)"
```

```
} else {
```

```
 Write-Host "Copie réussie."
```

```
}
```

```
Compression éventuelle
```

```
If ($Compress) {
```

```
 $zipFile = "$backupPath.zip"
```

```
 Compress-Archive -Path $backupPath -DestinationPath $zipFile -CompressionLevel
Optimal
```

```
 Remove-Item -Path $backupPath -Recurse -Force
```

```
 Write-Host "Compression effectuée : $zipFile"
```

```
 $savedPath = $zipFile
```

```
} else {
```

```
 $savedPath = $backupPath
```

```
}
```

```
Rotation : suppression des sauvegardes anciennes
```

```
$limitDate = (Get-Date).AddDays(-$RetentionDays)
```

```
Get-ChildItem -Path $DestinationRoot | Where-Object {
 $_.Name -match "^Backup_" -and $_.CreationTime -lt $limitDate
} | ForEach-Object {
 Remove-Item -Path $_.FullName -Recurse -Force
 Write-Host "Ancienne sauvegarde supprimée : $($_.Name)"
}

Write-Host "Sauvegarde terminée. Fichier : $savedPath" -ForegroundColor Green
...
```

## Étape 8 : Tester le script

```
```powershell  
# Créer un dossier source factice  
New-Item -Path C:\TestSource -ItemType Directory -Force  
"Fichier1" | Out-File C:\TestSource\doc1.txt  
"Fichier2" | Out-File C:\TestSource\doc2.txt  
  
# Exécuter sans compression  
.\BackupManager.ps1 -Source C:\TestSource -DestinationRoot C:\BackupTest -  
RetentionDays 1  
  
# Vérifier le contenu de C:\BackupTest\Backup_*  
  
# Exécuter avec compression  
.\BackupManager.ps1 -Source C:\TestSource -DestinationRoot C:\BackupTest -  
Compress  
  
# Attendre 1 minute, puis lancer une nouvelle sauvegarde pour voir la rotation (celle de  
plus d'1 jour sera supprimée)  
...
```